PROJECT APPROACH

This report describes the implemented application workflow, the reasoning behind its main design decisions, key assumptions and challenges, and improvements identified for future work.

Multimodal Document RAG

Approach, Methodology, and Design Rationale

Prepared: 30 October 2026

Scope: Streamlit application, PDF ingestion, hybrid retrieval, grounded responses

Executive summary

The project implements a local-first, PDF-focused retrieval-augmented generation (RAG) application. Users upload PDFs, explicitly start indexing, and ask questions in a Streamlit chat. The system extracts and chunks document content, stores embeddings and metadata in Chroma, retrieves candidate evidence through complementary search methods, reranks candidates, and generates an answer with source references. Conversation is routed separately from factual questions. The design prioritizes document provenance, duplicate-work avoidance, and explicit abstention when evidence is unavailable.

1. Goals and scope

The primary goal is to make information in uploaded PDF documents searchable and answerable through natural-language questions, while preserving the source document, page, and chunk used for each answer.

- Input: PDF files only, validated for file type, non-zero size, PDF signature, and a 100 MB per-file limit.
- Processing: Docling extraction, OCR and structure capture, type-aware chunking, embeddings, and persistent vector indexing.
- Interaction: Explicit sidebar processing control, visible per-file status, chat, follow-up context, citations, and a retrieved-chunk selector.
- Boundary: Extracted page imagery is retained as provenance/assets; this is not a full visual-language-model pipeline for interpreting every chart image.

2. Approach and methodology

2.1 Upload and duplicate control

Each uploaded file is read and hashed with SHA-256. Before expensive extraction, the pipeline searches ingestion manifests for a matching digest, then verifies that the corresponding chunk file and filtered Chroma records exist. A complete match is reported as already indexed and skipped. This avoids repeated processing of byte-identical PDFs while avoiding false skips when manifest, chunks, or vectors are missing.

2.2 Extraction and type-aware chunking

Docling converts the PDF into structured content and page-aware metadata. The chunking stage preserves text, tables, and image-related content as typed records, including headings, page numbers, and source regions where available. Chunk records are persisted as JSONL and support later retrieval and inspection.

2.3 Embedding and persistent storage

Text chunks are encoded with sentence-transformers/all-MiniLM-L6-v2 and stored in a persistent Chroma collection. Metadata includes document ID and name, chunk ID, content type, page range, headings, and available provenance. Persistence allows the application to reuse an index across runs without re-embedding unchanged documents.

2.4 Hybrid retrieval and reranking

For a factual request, the workflow searches each discovered document using semantic vector search and BM25 keyword search. Reciprocal-rank fusion combines the ranked lists; table-aware matching adds signals for row labels and headers. A configurable cross-encoder then reranks candidate query/chunk pairs and returns the top evidence. Combining methods is intended to handle both paraphrased questions and exact terms, figures, or table labels.

2.5 Intent, context, and answer generation

The LangGraph flow first contextualizes a follow-up when useful, using prior user questions rather than treating prior assistant claims as evidence. Greetings, personal conversation, and application-usage questions are handled separately. Factual or explanatory questions route to PDF retrieval, including requests that do not mention a PDF explicitly. Retrieved evidence is passed to the configured Gemini model with instructions to answer only from the evidence, cite document and page, and abstain for missing, unrelated, or contradictory support.

2.6 User interface and startup

The Streamlit sidebar provides a deliberate Process and index PDFs action and progress/status feedback. Questions are added to chat only after an answer has been generated. Embedding and reranker model objects are warmed and cached at app startup so weights are reused across Streamlit reruns and queries.

3. Design decisions and rationale

- PDF-first scope: A narrow, explicit input contract supports reliable validation, extraction, page references, and a manageable user workflow.

- Explicit processing action: Indexing can be expensive. Requiring a button makes the transition clear, exposes progress, and prevents a query from racing pending uploads.

- Content hashing: A cryptographic digest identifies identical file bytes independent of filename; checking manifest, chunks, and Chroma together prevents treating incomplete work as a successful duplicate.

- Typed chunks and metadata: Separating text, table, and image-related records retains useful structure and enables the UI to show the actual retrieved passage with provenance.

- Hybrid search: Dense retrieval captures semantic similarity; BM25 and table heuristics help with exact names, values, and row/column clues. Reranking improves ordering after candidate retrieval.

- Separate conversation routing: Greetings and app-help do not need document search. Factual questions should be checked against the corpus even when no fixed keyword is present.

- Grounded generation and abstention: The model receives retrieved evidence rather than being asked to answer from general knowledge. Missing or weak support should result in an explicit limitation, not an invented answer.

- Local persistence and model caching: Chroma and downloaded model caches avoid unnecessary rebuilds; Streamlit resource caching avoids repeated in-memory model initialization on reruns.

- Environment template, not secrets: Configuration is documented in .env.example with empty credential slots; real credentials are excluded from source control.

4. Assumptions

- Users have the right to process and share their uploaded PDFs, and the documents are valid, readable PDFs.

- Local storage is available for extracted assets, JSONL chunks, model caches, and the persistent vector database.

- Model downloads may be required on first use; a Gemini API key is optional but needed for generative answers and model-assisted intent/context handling.

- Page citations reflect extraction metadata. OCR and chart text may omit spatial relationships or contain recognition errors.

- Content-hash deduplication detects identical bytes only; re-saved copies with different PDF metadata may be processed again.

- The current local app is designed for a single-user workspace and does not provide production-grade multi-user authorization or tenant isolation.

5. Challenges encountered

- Chroma API compatibility: The installed Collection.count() did not accept a where filter. Duplicate checks were changed to use filtered get() and count returned IDs.

- Model startup latency: Embedding and reranking weights loaded repeatedly during reruns/queries. Shared cached factories and Streamlit startup warming were added.

- Windows model cache behavior: Hugging Face may warn about unsupported symlinks; cache downloads still work but can consume more disk space.

- Tables and visual evidence: Flattened OCR text may preserve numbers and labels but lose their layout, so a chart value cannot safely be attributed to a label without explicit evidence.

- Output formatting: LLM output can include malformed Markdown such as stray backticks around currency amounts; answer formatting instructions and narrow cleanup were added.

6. Validation and current limitations

Automated tests cover intent routing, conversation history, source metadata, answer parsing and provider errors, duplicate PDF checks, ingestion orchestration, reranker ordering/fallback, and model-cache reuse. The test suite reported 29 passing tests after the model caching changes. This demonstrates covered unit behavior, not a complete real-world evaluation of extraction quality or answer correctness.

The implementation remains dependent on extraction quality, retrieval relevance, and the configured model. An instruction to abstain reduces unsupported claims but cannot guarantee that a generative model will never hallucinate. The no-key fallback is intentionally limited and cannot replace general-purpose conversation or semantic relevance judgment.

The current visual pipeline tracks extracted image/page assets but does not consistently reason over chart geometry. The model weights are not part of the source repository and must be fetched or populated in the local Hugging Face cache. A fresh end-to-end test over a representative PDF is still valuable after deployment or model/dependency changes.

7. Improvements considered

- Evidence relevance gate: Add calibrated retrieval confidence and/or a dedicated evidence-sufficiency classifier, evaluated on relevant and deliberately unrelated questions, before generation.
- Citation validation: Require structured model output that maps answer claims to specific source chunks/pages, then validate references against the evidence actually retrieved.
- Chart and table reasoning: Add layout-aware table reconstruction and optional vision-model interpretation of page crops, preserving visual coordinates and checking numerical relationships.
- Quality evaluation: Build a representative benchmark of PDFs and questions with expected source pages; measure retrieval recall, citation accuracy, answer support, and abstention on unanswerable questions.
- Operational hardening: Add structured logging, user-facing model/provider error states, resource limits, index health checks, and explicit backup/restore guidance for persistent data.
- Multi-user deployment: If deployed beyond a trusted local environment, add authentication, per-user document authorization, tenant-scoped indexes, upload scanning, and data-retention controls.
- Cache controls: Expose model cache location and startup status, and offer configuration to disable the local reranker on constrained hardware.

8. End-to-end flow

- Upload PDF -> validate size/signature -> compute SHA-256.
- Check manifest, chunk file, and Chroma vectors -> skip only if complete duplicate.
- Extract PDF structure/OCR -> create typed chunks with source metadata.
- Embed chunks -> persist vectors and metadata in Chroma.
- Classify user message -> conversation path or factual RAG path.

- Retrieve with vector + BM25 + table signals -> rerank candidates.
- Generate only from evidence -> return answer, citations, and inspectable source chunks.

Conclusion

The solution combines a transparent PDF ingestion workflow with provenance-aware hybrid retrieval and a chat interface. Its core design favors inspectable evidence, reproducible local indexing, and explicit uncertainty. The most valuable next steps are stronger evidence-sufficiency validation, layout-aware chart/table interpretation, and systematic evaluation against representative documents.